AWS Order Request Queue & Processing System

Week 1 — Requirements Analysis and Initial Architecture

Project Type: Implementation / Architecture Design

Duration: 4 Weeks

Business Context: QuickCart GmbH — Online Order Processing

AWS Focus: Amazon SQS, AWS Lambda, IAM, Amazon CloudWatch, optionally API Gateway

Implementation Approach: Cost-conscious, serverless AWS architecture

Purpose

This report defines the requirements, assumptions, AWS service choices, architecture alternatives, initial diagram, and message flow for a reliable order-processing system. Week 1 focuses on planning and design; no resources are deployed and no application code is written during this phase.

1. Project Overview

QuickCart GmbH is an online store that may receive a large number of orders during promotional events. If the order-processing service becomes slow or temporarily unavailable, a synchronous system may delay customer requests or fail to handle the increased workload.

This project proposes an asynchronous order-processing system using Amazon SQS and AWS Lambda. Orders are placed in a durable queue and processed independently of the customer request. This helps absorb traffic spikes, supports retries, and reduces the risk of losing accepted orders when processing is temporarily unavailable.

Week 1 objective: Define requirements, compare architectural options, evaluate AWS services, and prepare an initial architecture without deploying resources or writing application code.

2. Business and Technical Requirements

2.1 Business Requirements

- Accept customer orders during normal and peak traffic.
- Avoid losing accepted orders when processing is temporarily unavailable.
- Allow customers to submit orders without waiting for order processing to finish.
- Automatically retry failed processing and isolate repeatedly failing messages.
- Monitor queue activity and processing errors.
- Keep the solution secure, simple, and cost-conscious.

2.2 Technical Requirements

- Use Amazon SQS as the durable buffer between order submission and processing.
- Use AWS Lambda to validate incoming orders and process queued messages.
- Configure a Dead-Letter Queue (DLQ) for messages that repeatedly fail.
- Use AWS IAM roles and policies to restrict access.
- Use Amazon CloudWatch for logs, metrics, and troubleshooting.
- Design the system so it can handle traffic spikes and be extended later.

3. Assumptions and Constraints

Assumptions

- Orders are submitted through a simple API.
- Temporary processing delays are acceptable, but accepted messages should remain available.
- Strict message ordering is not required for the initial version.
- The project is a small educational system with limited expected traffic.

Constraints

- The design should consider AWS Free Tier eligibility and potential service charges.
- Week 1 covers planning only; deployment and implementation will follow later.
- Lambda execution limits, duplicate message delivery, and retry behavior must be considered.
- A VPC Endpoint is optional and will be evaluated based on networking needs and cost.

4. Synchronous vs. Asynchronous Processing

In a synchronous design, the customer request waits for the processing operation to finish. If the processing service is slow or unavailable, the request may be delayed or fail.

In an asynchronous design, the submission service places the order message in Amazon SQS and returns a response once the queue accepts it. A separate Lambda function processes the message later.

For QuickCart, asynchronous processing is recommended because it separates order submission from processing, absorbs temporary traffic spikes, and allows queued messages to wait until processing becomes available. A successful submission response confirms queue acceptance, not completion of order processing.

5. AWS Service Analysis

AWS Service	Purpose and Reason for Selection
Amazon API Gateway (HTTP API)	Provides an API endpoint for order submissions and invokes the submission Lambda. HTTP API is a simple option for this project.
AWS Lambda	Runs the submission and processing logic without managing servers.
Amazon SQS Standard	Temporarily stores order messages, decouples services, and absorbs traffic spikes. Standard queues support high throughput but may deliver messages more than once and do not guarantee strict ordering.
Amazon SQS Dead-Letter Queue (DLQ)	Holds messages that fail repeatedly so they can be investigated without continually blocking normal processing.
AWS IAM	Grants each component only the permissions it needs.
Amazon CloudWatch	Collects logs and metrics to help identify errors and monitor queue behavior.
Amazon VPC / SQS VPC Endpoint	The Default VPC may be used for Lambda networking if required. An SQS VPC Endpoint is optional for private connectivity and must be assessed against cost and network requirements.

Alternatives considered

SQS FIFO provides ordering and deduplication features but is not necessary when strict ordering is not required. A database such as DynamoDB could store order details and status for long-term access, but it is outside the initial scope because SQS is the processing queue, not the permanent order database.

6. Architecture Options

Option A — Direct Queue Submission

The order client sends messages directly to Amazon SQS, and a Lambda function processes them. This design is simple and has fewer components, but direct client access requires careful security configuration and is less suitable when a custom API and server-side validation are required.

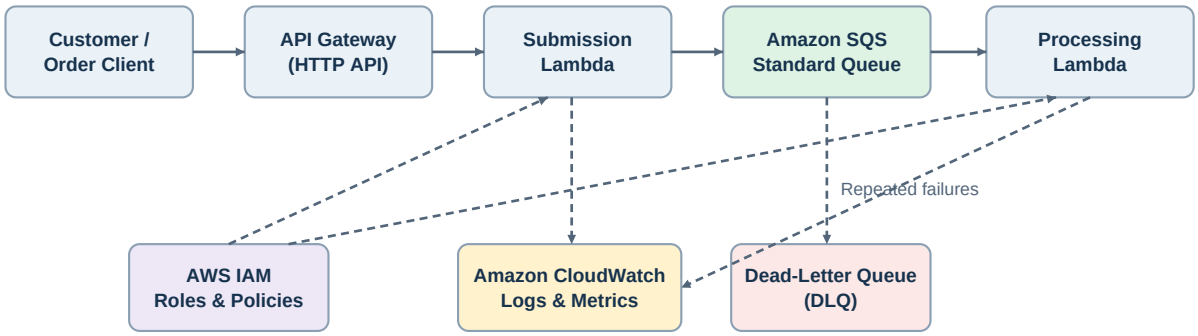
Option B — API-Based Submission (Recommended)

The customer submits an order through API Gateway. A Submission Lambda validates the request and sends the message to SQS. A separate Processing Lambda consumes messages from the queue. A DLQ stores repeatedly failing messages.

Option B is recommended because it provides a clear API boundary, server-side validation, controlled access to SQS, and independent processing. It includes more components than Option A, so its cost and configuration should be monitored.

7. Initial Architecture Diagram

The diagram presents the proposed order flow and the supporting AWS services. The Default VPC is shown as a planned networking environment for the Lambda functions, not as an already deployed resource. An SQS VPC Endpoint may be added if private connectivity is required; the final network configuration will be decided during implementation.



Default VPC is a possible Lambda networking environment; API Gateway and SQS are not inside the VPC. An SQS VPC Endpoint is optional and depends on the final network design.

8. Initial Message Flow

- The customer submits an order through API Gateway.
- API Gateway invokes the Submission Lambda.
- The Submission Lambda validates the request and sends the order message to SQS.
- After SQS accepts the message, the API returns a submission confirmation.
- The Processing Lambda receives and processes the message asynchronously.
- If processing fails, the message can be retried. After the configured number of unsuccessful receives, SQS moves it to the DLQ according to the redrive policy.
- CloudWatch collects logs and relevant metrics for monitoring and troubleshooting.

Because SQS Standard may deliver a message more than once, the processing logic should be idempotent so that duplicate deliveries do not cause duplicate business actions.

9. Security, Reliability, and Cost

Security: Use separate IAM roles with least-privilege permissions. The submission function should be allowed to send messages to the main queue, while the processing function should have only the permissions it needs to consume messages and write logs. Protect the API and validate incoming orders.

Reliability: Keep accepted messages in SQS until they are successfully processed and deleted. Configure retries and a DLQ. Monitor queue depth, message age, and processing errors. Queue acceptance alone does not guarantee that business processing has completed, so persistent order records may be added in a future version if required.

Cost: Keep the design small and avoid unnecessary services. Review Lambda invocations, API Gateway requests, SQS usage, CloudWatch logs, and any VPC Endpoint charges. The Free Tier depends on account eligibility and current AWS terms; actual costs should be checked before deployment.

10. Conclusion and Week 1 Deliverables

The proposed architecture uses API Gateway, Lambda, SQS Standard, a DLQ, IAM, and CloudWatch to support reliable asynchronous order processing. It separates customer submissions from processing, handles temporary backlogs, and provides a way to investigate repeated failures.

Week 1 establishes the business and technical requirements, assumptions, synchronous-versus-asynchronous comparison, AWS service analysis, two architecture options, an initial diagram, and the message flow. Implementation and detailed network configuration will be addressed in later weeks.